Analytics Engineering: Session 4

Modeling Layers, Lineage & Business Logic

Agenda

- The DRY principle and why we build in layers
- The four layers: source/seed → staging → intermediate → mart
- The DAG and lineage in dbt
- Using lineage to navigate and debug your project
- Business logic in SQL — patterns and examples
- Python models (extra)
- Hands-on exercise

Where we left off

You finished session 3 with:

- `sources.yml` mapping every raw table
- A `segments.csv` seed loaded
- One staging model (`stg_*`) per source table
- `unique` / `not_null` tests on staging primary keys

Today: what comes after staging, and how to think about it.

The DRY principle

"Don't Repeat Yourself"

- If you write the same SQL logic twice, refactor it.
- **Modularity:** build a model once, reference it (`{{ ref() }}`) many times.
- **Benefits:**
  - **Maintainability** — fix a bug in one place, it propagates.
  - **Consistency** — everyone uses the same definition of *revenue*.

Why layers?

Going straight from raw sources to a final business table mixes too many concerns:

- Cleaning + joins + business rules + aggregations all in one query.
- Hard to test, hard to debug, impossible to reuse.

dbt models the journey in layers — each layer has one job.

The four layers

Layer 0: Source & Seed

Source — raw data already in the warehouse, loaded by an EL tool.

- Defined in `sources.yml`, referenced as `{{ source('raw', 'orders') }}`.
- We never transform a source in place.

Seed — small static CSVs in `seeds/`.

- Loaded with `dbt seed`, referenced as `{{ ref('segments') }}`.
- Use for lookup tables, mappings, country codes — *not* for raw business data.

Layer 1: Staging

Goal: a reliable, clean foundation.

- **One staging model per source table** (1:1 mapping).
- Materialized as **views** by default — cheap to rebuild.
- Tasks: rename, cast types, light cleaning (trim, null handling).

Anti-patterns: no joins, no aggregations, no business rules.

Staging — example

```sql
-- models/staging/stg_orders.sql
select
    order_id,
    customer_id,
    order_date::date            as order_date,
    status,
    total_amount::numeric(10,2) as total_amount
from {{ source('raw', 'orders') }}
```

Downstream models always reference `{{ ref('stg_orders') }}`, never the raw source.

Layer 2: Intermediate

Goal: handle complexity, isolate reusable logic.

- **Logic-concentric** — bridges staging and marts.
- Tasks: joins, calculated fields, deduplication, fan-out fixes.
- **Internal:** end users and BI tools should not query these directly.
- Naming: `int_<descriptive>` (e.g., `int_orders_with_items`).

If two marts need the same join, push it up into an intermediate.

Layer 3: Marts

Goal: business-ready, easy to query.

- The tables analysts and BI tools actually consume.
- Materialized as **tables** (sometimes incremental — later sessions).
- Naming convention you'll see in this course:
  - `dim_<entity>` — descriptive context (`dim_customers`, `dim_products`).
  - `mart_<topic>` — measurable events or aggregates (`mart_orders`, `mart_revenue_by_segment`).

Pick the name that best describes the table. The layer is what matters, not the prefix.

A worked example

```
stg_customers ─┐
stg_orders ────┼──> int_orders_enriched ──> mart_orders
stg_order_items ──> int_order_items_summary ─┘
                                              └─> mart_revenue_by_segment
```

Each model has **one job** and a clear set of upstream dependencies.

The DAG

Directed Acyclic Graph — the map of your project.

- **Directed:** data flows one way, source → mart.
- **Acyclic:** no loops; a model cannot depend on itself, directly or transitively.
- **Graph:** nodes are models, edges are `ref()` and `source()` calls.

dbt builds the DAG automatically by parsing your `ref()` and `source()` calls — you never write it by hand.

Lineage in dbt

Two ways to view the DAG:

- **VS Code dbt extension** → click **Lineage**.
- **dbt docs site:** `dbt docs generate && dbt docs serve` → interactive graph in the browser.

What you see:

- Every source, seed, and model as a node.
- Arrows for every `ref()` / `source()` dependency.
- Click a node → its description, columns, and tests.

Reading lineage to debug

When a model is wrong, lineage tells you **where to look**:

- **Upstream** of a broken model → possible cause.
- **Downstream** of a changed model → impact radius.

Common questions lineage answers:

- "If I change `stg_orders`, what breaks?"
- "Where does `customer_segment` originally come from?"
- "Which marts depend on this seed?"

Selecting models with lineage

dbt's selector syntax mirrors lineage:

```
dbt build -s stg_orders          # just this model
dbt build -s stg_orders+         # this model and everything downstream
dbt build -s +mart_orders        # this model and everything upstream
dbt build -s +mart_orders+       # full upstream and downstream
dbt build -s staging             # everything in the staging folder
dbt build -s tag:finance         # everything tagged "finance"
```

Use these constantly while developing. Don't rebuild the whole project for one model.

Business logic in SQL

The interesting work happens in **intermediate** and **mart** models.

A few patterns you will use over and over:

1. Cleaning & casting (staging)
2. `case when` classification
3. Joins (intermediate)
4. Aggregations (marts)
5. Window functions (rankings, running totals)

Pattern 1 — cleaning & casting

```sql
-- stg_customers.sql
select
    customer_id,
    trim(lower(email))           as email,
    initcap(first_name)          as first_name,
    initcap(last_name)           as last_name,
    coalesce(country, 'unknown') as country,
    customer_segment
from {{ source('raw', 'customers') }}
```

One column, one cleaning step. Easy to read, easy to test.

Pattern 2 — `case when` classification

```sql
-- int_orders_classified.sql
select
    order_id,
    total_amount,
    case
        when total_amount ≥ 500 then 'high'
        when total_amount ≥ 100 then 'medium'
        else 'low'
    end as order_size_bucket
from {{ ref('stg_orders') }}
```

Encode the business rule once — every downstream model agrees on the buckets.

Pattern 3 — joins (intermediate)

```sql
-- int_orders_enriched.sql
select
    o.order_id,
    o.order_date,
    o.total_amount,
    c.customer_segment,
    c.country
from {{ ref('stg_orders') }}    as o
left join {{ ref('stg_customers') }} as c
    using (customer_id)
```

One row in, one row out — preserve grain. Comment why you chose `left` *vs* `inner`*.*

Pattern 4 — aggregation (marts)

```sql
-- mart_revenue_by_segment.sql
select
    customer_segment,
    date_trunc('month', order_date) as order_month,
    count(*)            as orders,
    sum(total_amount)   as revenue,
    avg(total_amount)   as avg_order_value
from {{ ref('int_orders_enriched') }}
group by 1, 2
```

Marts answer questions. The grain (segment, month) is the whole point.

Pattern 5 — window functions

```sql
-- int_customer_orders_ranked.sql
select
    customer_id,
    order_id,
    order_date,
    total_amount,
    row_number() over (
        partition by customer_id order by order_date
    ) as order_rank,
    sum(total_amount) over (
        partition by customer_id order by order_date
    ) as customer_running_total
from {{ ref('stg_orders') }}
```

Use windows for "first order", "last touch", running totals, lead/lag.

Python models (extra)

When SQL is awkward — complex string parsing, ML, calling an API — dbt also supports **Python models**.

```python
def model(dbt, session):
    dbt.config(materialized="table")
    df = dbt.ref("dim_customers")
    df = df.filter(df["country"] == "ES")
    return df
```

Same `ref()`, same DAG, same tests. Requires a Python-capable warehouse (DuckDB, Snowflake, BigQuery). docs

What we covered

- Why we model in layers — DRY, modularity, testability.
- The four layers: **source/seed → staging → intermediate → mart**.
- The DAG and how dbt builds it from `ref()` calls.
- Reading lineage to navigate and debug.
- Five SQL patterns you will keep reusing.
- Python models exist — we'll see them again later.

Hands-on exercise

Build your first intermediate model

Setup check

You should already have, from session 3:

- All staging models (`stg_*`) for every source table.
- The `segments` seed loaded.
- `unique` / `not_null` tests on staging primary keys.

If anything is missing, fix that first — the rest of today's exercise depends on it.

```
dbt build -s staging
```

Exercise: build `int_orders_enriched`

Create `models/intermediate/int_orders_enriched.sql`. Join `stg_orders` with `stg_customers`.

Expected output: one row per order, with these columns:

- `order_id`, `order_date`, `status`, `total_amount`
- `customer_id`, `customer_segment`, `country`

Constraints:

- Use `{{ ref(...) }}` — never query staging tables directly.
- Use a `left join` from orders → customers.
- No aggregations, no `group by`.

Add tests

Create `models/intermediate/int_orders_enriched.yml`:

```yaml
version: 2
models:
  - name: int_orders_enriched
    columns:
      - name: order_id
        tests:
          - unique
          - not_null
      - name: customer_id
        tests:
          - not_null
          - relationships:
              to: ref('stg_customers')
              field: customer_id
```

Verify with lineage

1. Open the **Lineage** view (VS Code extension or `dbt docs serve`).
2. Find `int_orders_enriched`.
3. Check that:
   - It depends on **two** staging models, not on raw sources directly.
   - The arrows match your `ref()` calls.
4. Sanity-check the row count:

```
select count(*) from {{ ref('int_orders_enriched') }}
-- should equal:
select count(*) from {{ ref('stg_orders') }}
```

Stretch goal (if time)

Build `mart_revenue_by_segment` on top of `int_orders_enriched`:

- Group by `customer_segment` and `date_trunc('month', order_date)`.
- Compute `orders`, `revenue`, `avg_order_value`.
- Add `not_null` tests on the grouping columns.
- Run `dbt build -s +mart_revenue_by_segment` and inspect the lineage.

Next session: Practice I — Building the Foundation (full DAG end-to-end).